

Clase 15

Autenticación y autorización

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl

1. E0 es hasta mañana, luego E1
2. Votemos por sus dibujos

Anuncios

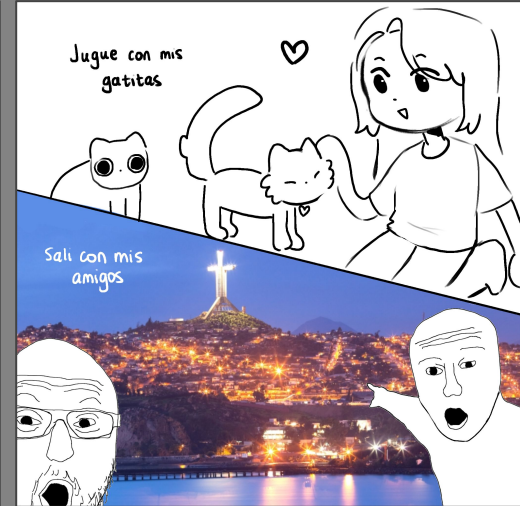
A votar por un dibujo



1



2



3

1

2

3

4

5

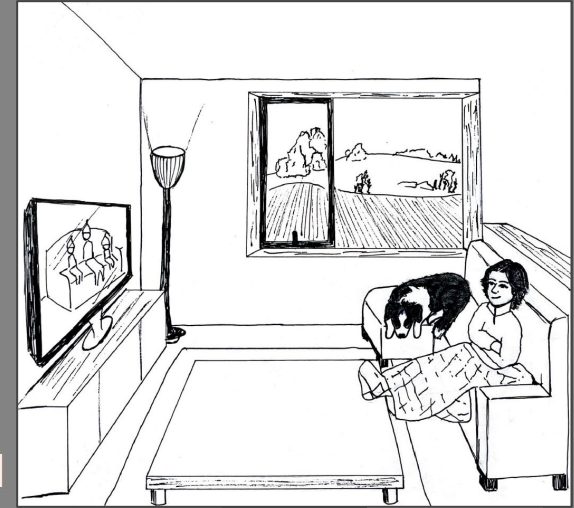
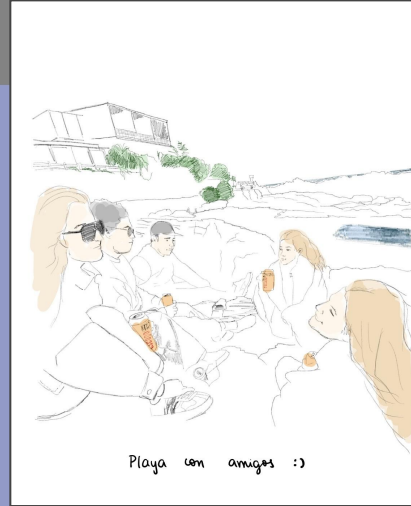
6



3

26

A votar por un dibujo



1

2

3

4

5

6

4

5

6



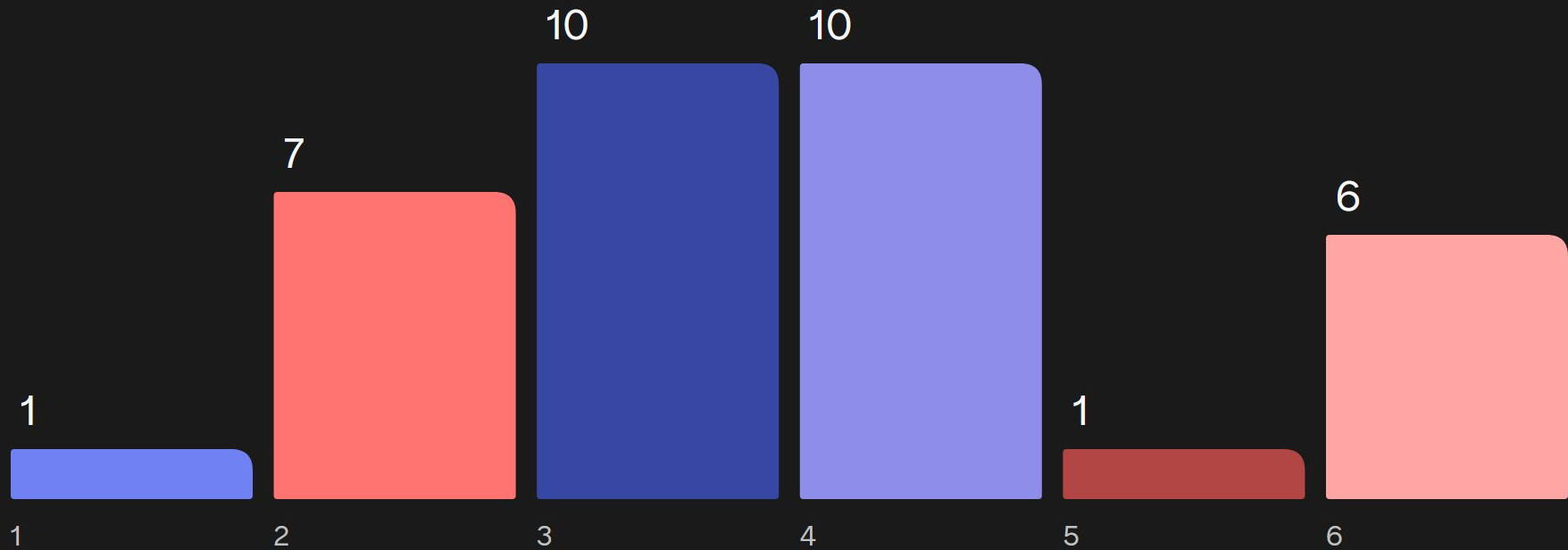
3



26



A votar por un dibujo



A votar por un dibujo



1

4

5

6

1

2

3

4

5

6



3



26



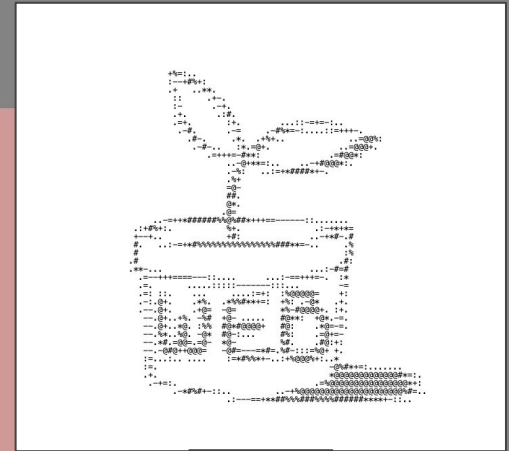
A votar por un logo



1



2



3

1

2

3

4

5

6



3

26

A votar por un logo

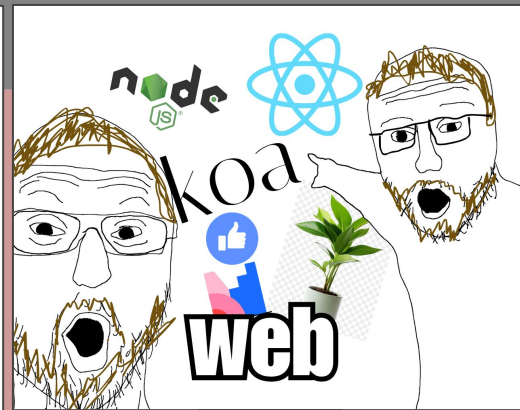
9



4



5



6

1

2

3

4

5

6



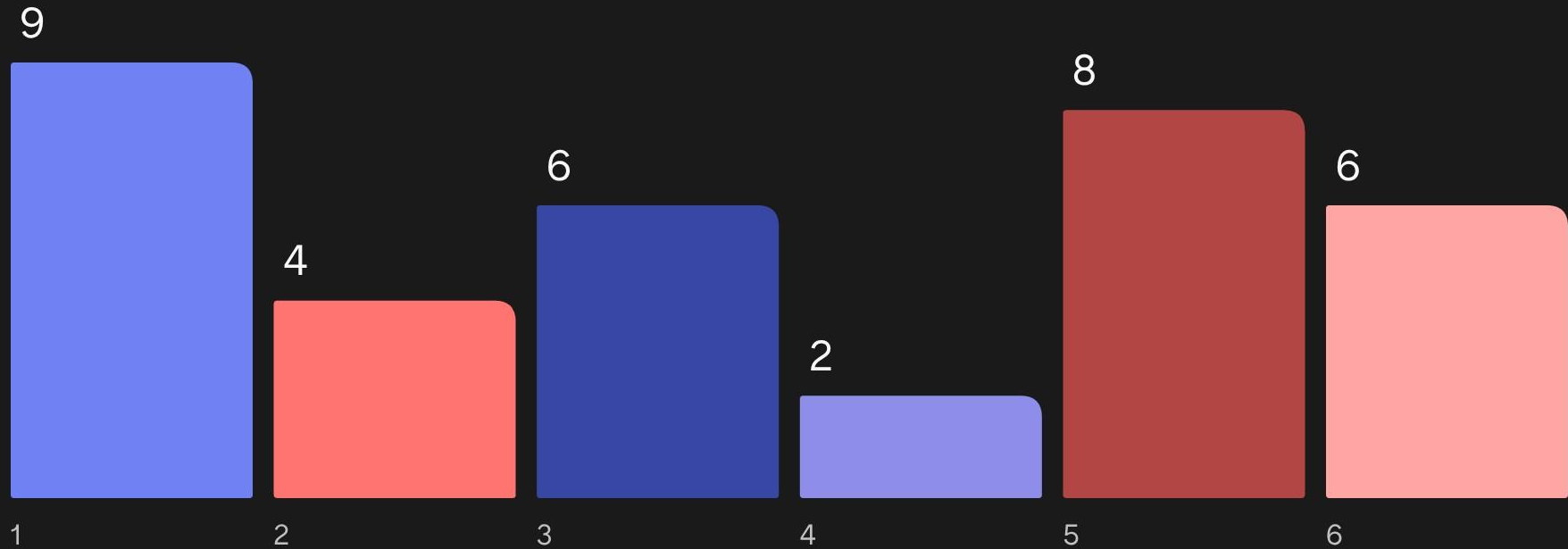
3



26



A votar por un logo

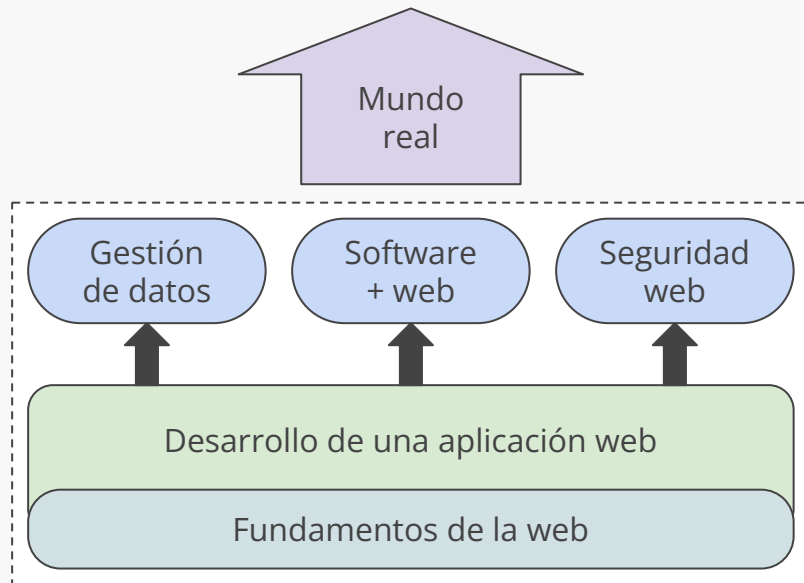


A votar por un logo



Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

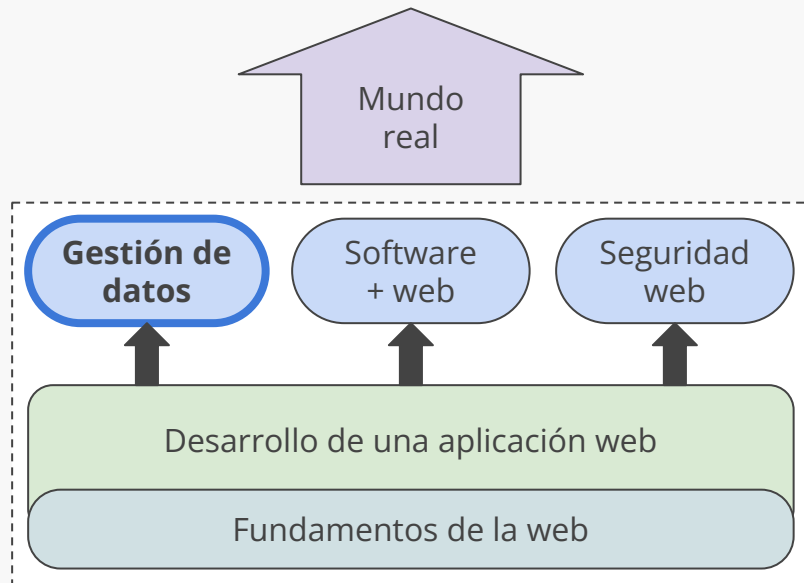


Contenidos del curso

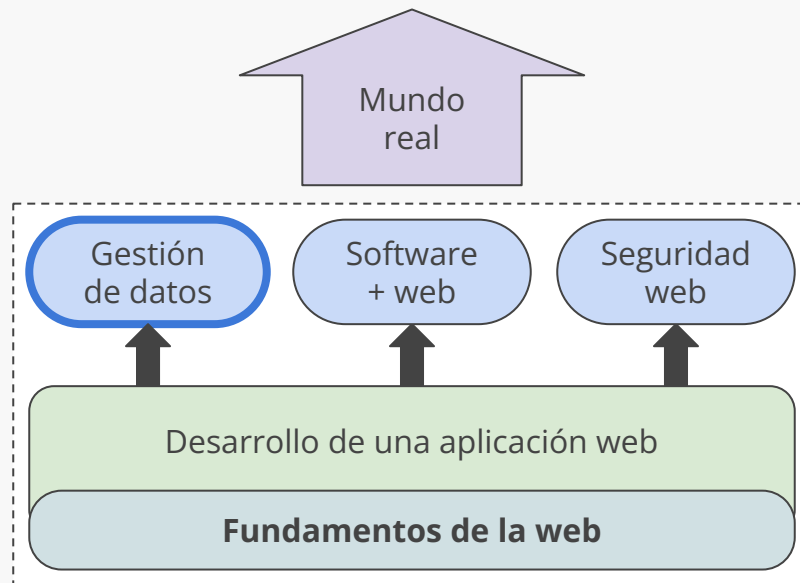
Gestión de datos

Veremos cómo diseñar y manejar **cookies**, **sesiones** y **otros datos** en el navegador...

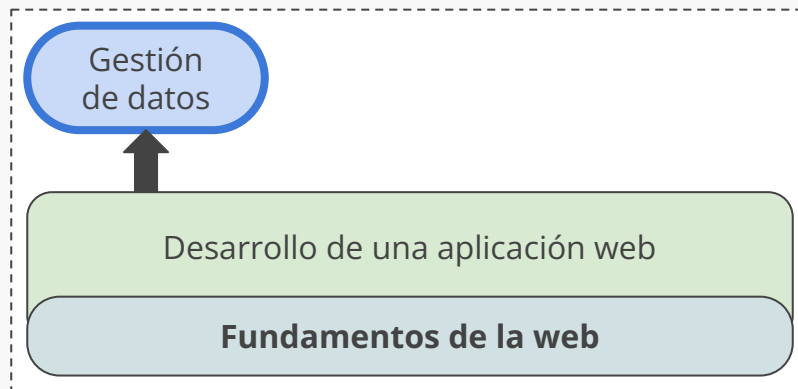
... para habilitar **funcionalidades avanzadas** de manera segura y con técnicas modernas



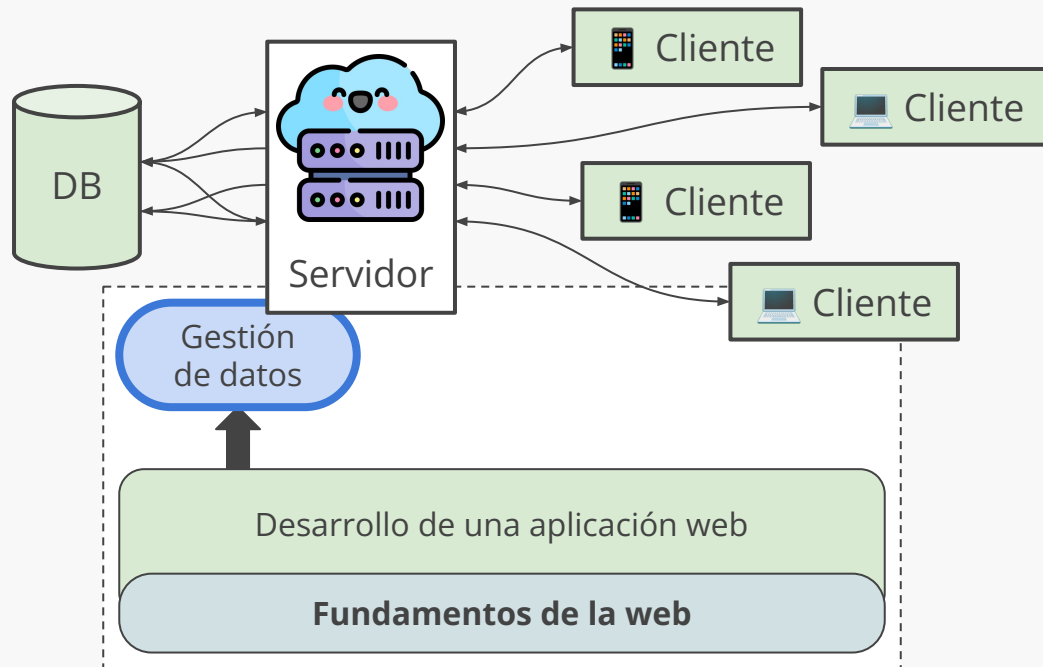
Contenidos del curso



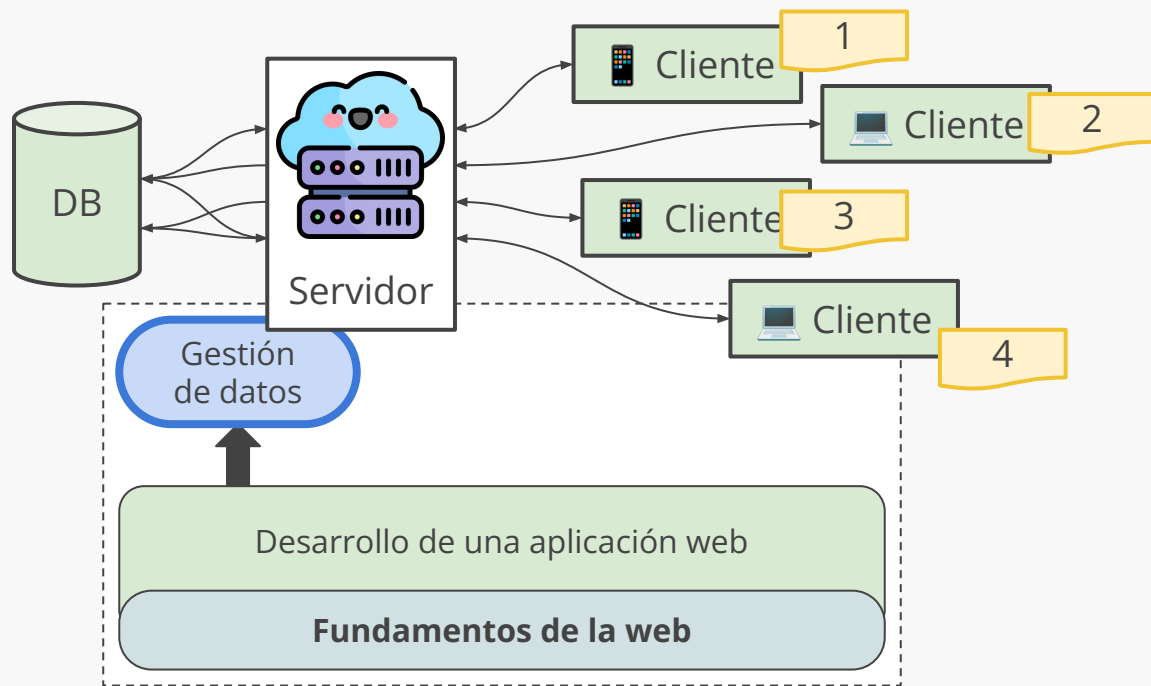
Contenidos del curso



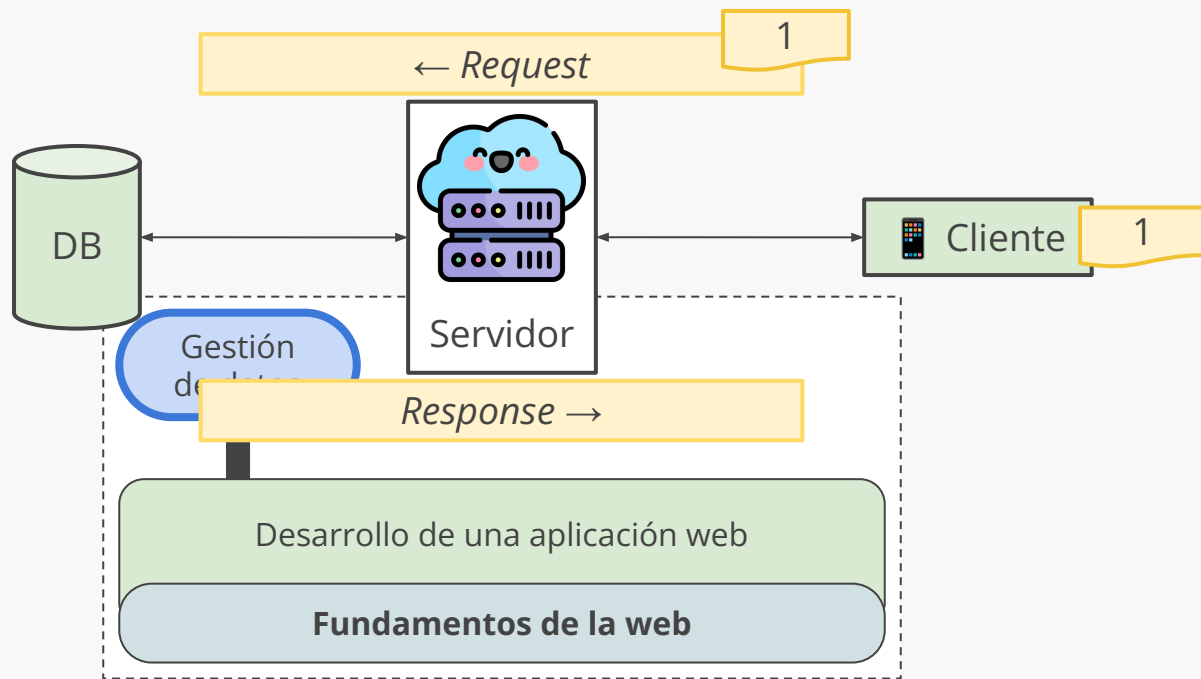
Motivación



Motivación



Motivación



Motivación



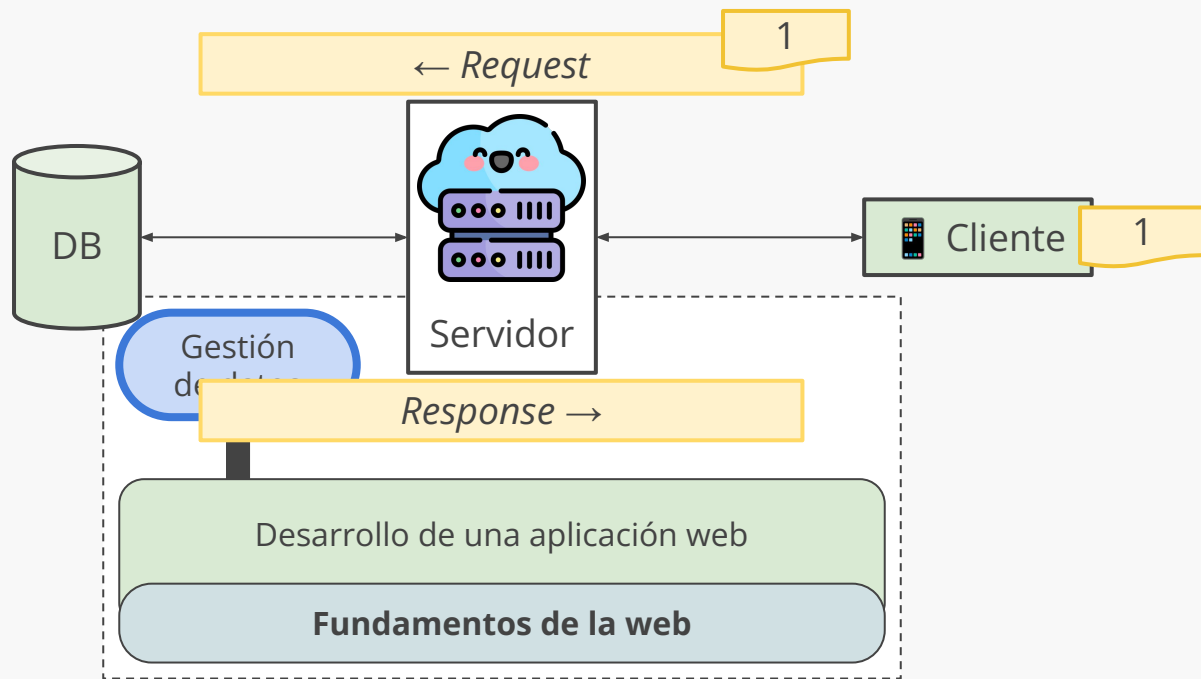
Objetivo de la clase de hoy

- ❑ **Entender** la importancia del concepto de autorización
- ❑ **Inferir** cómo se implementan distintos mecanismos de inicio de sesión
- ❑ **Identificar** buenas prácticas para el manejo de claves y sesiones

¿Existen más formas de "iniciar sesión"?

- Una vez que un usuario se ha identificado, le asignamos un *string* (ya sea *token*, *session_id* u otra cosa) que utilizaremos para **validar que el usuario es quien dice ser** en cada *request*
- En la práctica, esta identificación puede venir **por medio de la clave del usuario o por medio de alguna tercera parte confiable** (por ejemplo, si el usuario inicia sesión con su cuenta de Google)
- La implementación de estos mecanismos se construye sobre **estándares bien definidos** (como SAML, OAuth y OpenID Connect)

Motivación



Agenda



Identificación y autenticación

Hashing de contraseñas

JSON Web Tokens (JWT)

Open Authorization (OAuth)

Single Sign-On (SSO)

Autenticación vs autorización

Dos términos se suelen confundir entre sí son autenticación y autorización:

- **Autenticación**: verificar que el usuario es quien dice ser
 - Formulario de *login*
 - Autenticación vía HTTP
- **Autorización**: decidir si un usuario puede acceder a un recurso
 - Control de acceso a URLs
 - Identificación de permisos

Notar que **si no hay autenticación no puedo determinar autorización**

Autenticación

- **Objetivo:** Verificar que el usuario es quien dice ser
- Tradicionalmente se utilizan uno o dos de los siguientes 3 factores:
 - Algo que el usuario **sabe** (ej: una contraseña)
 - Algo que el usuario **tiene** (ej: un celular o una llave criptográfica)
 - Algo que el usuario **es** (ej: huella digital o un dato biométrico)

Mientras más factores se utilicen, más seguros estaremos de que el usuario que es quien dice ser (por eso existe *multi-factor authentication*, MFA)

Consejos para implementación

- Nombres de usuario deben almacenarse de forma **case insensitive**
aaossa es lo mismo que AAossa o AaOssa
- Nombres de usuario deben ser **únicos**
- Colocar **requerimientos mínimos** pero razonables a las contraseñas
Uso de caracteres especiales y un mínimo de largo son útiles
- El principal factor para que una contraseña sea segura es el **largo**
Contraseñas muy cortas son fácilmente “adivinables”
- Forzar cambios regulares de contraseña lleva a contraseñas débiles

Comentarios para implementación

- Nombres de usuario deben almacenarse de forma *case insensitive*
aaossa es lo mismo que AAossa

Pregunta# ¿Cuáles malas prácticas encontramos en el mundo real?

Uso de caracteres especiales y un mínimo de largo son útiles

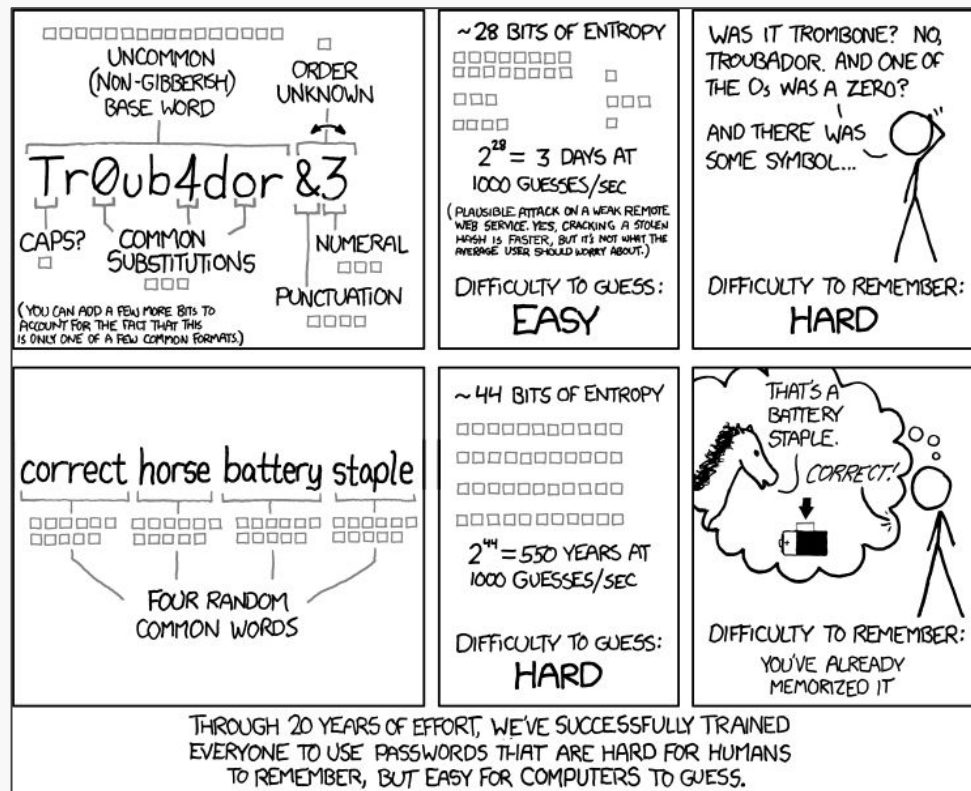
- El principal factor para que una contraseña sea segura es el **largo**
Contraseñas muy cortas son fácilmente “adivinables”
- Forzar cambios regulares de contraseña lleva a contraseñas débiles

¿Qué es una contraseña débil?

Como el espacio de caracteres posibles es finito, podemos pensar cada contraseña como una aguja en un pajar...

la pregunta correcta es: ¿qué tan difícil es encontrar la contraseña correcta en el "pajar"?

Password's Haystack



Have I Been Pwned:

Sitio para comprobar si sus datos personales han aparecido en brechas de datos (de Troy Hunt)

[Home](#)[Notify me](#)[Domain search](#)[Who's been pwned](#)[Passwords](#)[API](#)[About](#)[Donate](#)  

';--have i been pwned?

Check if your email address is in a data breach

pwned?

Using Have I Been Pwned is subject to [the terms of use](#)

884

pwned websites

14,953,859,802

pwned accounts

115,799

pastes

229,165,825

paste accounts

Mozilla Monitor:

Sitio para comprobar si sus datos personales han aparecido en brechas de datos (de Mozilla)



[Sign In](#)

Find where your private info is exposed — and take it back

We scan data breaches to see if your data has been leaked and give you steps to fix it.

yourname@example.com

[Get free scan](#)

Enter your email address to check for data breach exposures.



Data breaches happen every 11 minutes, exposing your private information — but don't worry, we can help.

Agenda

Identificación y autenticación



Hashing de contraseñas

JSON Web Tokens (JWT)

Open Authorization (OAuth)

Single Sign-On (SSO)

Encriptación vs *Hashing*

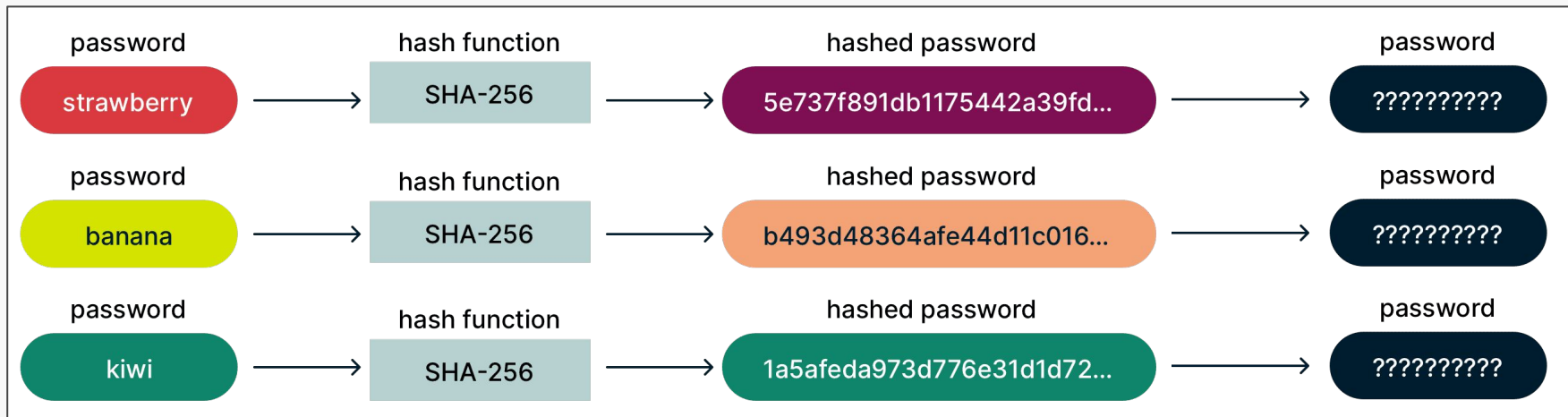
Si bien tanto **encriptación** como ***hashing*** son términos relacionados con protección de información, ambos lo hacen en maneras distintas:

- **Encriptar** codifica los datos de manera que puedan ser **desencriptados (decodificados) con una *key* o clave**. La intención es pasar información a otra entidad y que el receptor pueda usar las *keys* apropiadas para desencriptar y acceder a los datos. Un ejemplo de encriptación: HTTPS ([Ver más](#))
- ***Hashing*** también codifica los datos, pero el objetivo es probar su autenticidad. No existe una *key* o clave de descifrado (o “dehashing”) que permita recuperar los datos originales. Un ejemplo de *hashing*: contraseñas, identificadores de *commits* (SHA1-)

En general, encriptamos datos cuando queremos enviar la información a algún destino y usamos *hashing* cuando queremos almacenar la información en algún lugar

Hashing de contraseñas

Una función de *hashing* transforma cualquier *string* en otro *string* aparentemente aleatorio, y a diferencia de un proceso de encriptación, no hay proceso reverso (es “one-way”)

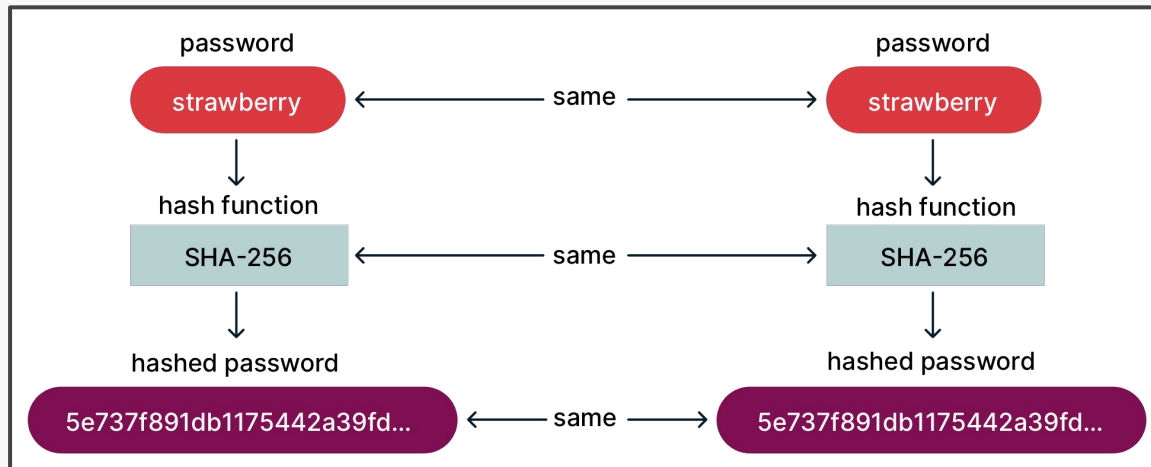


bcrypt

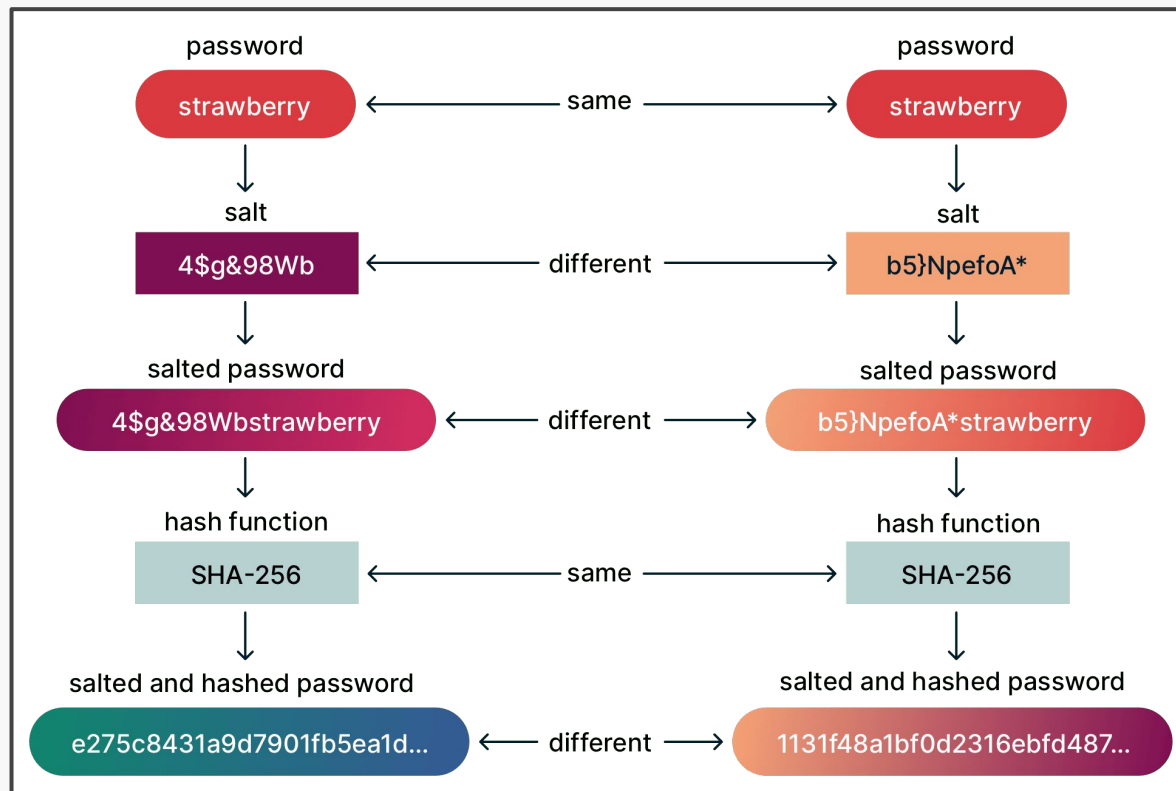
bcrypt es una función de *hashing* para contraseñas desarrollado en 1999 por Niels Provos y David Mazières. Algunas de las medidas de seguridad que incluye son:

- Mecanismo de *salt* incorporado para proteger contra ataques tipo [rainbow table](#) (tabla precomputada con los *hashes* resultantes de una función de *hash*)
- Incluye un factor de costo que determina la cantidad de rondas de *hashing*, permitiendo aumentar el esfuerzo computacional
- Es lento de calcular intencionalmente, para dificultar intentos de ataque de fuerza bruta que se prueben contra el algoritmo
- Es adaptable en el tiempo, permitiendo aumentar el factor de costo a medida que el hardware disponible también mejora

(Agregando *salt*)



(Agregando *salt*)



¿Cómo se ve un hash de bcrypt?

El input para bcrypt es una contraseña (de hasta 72 bytes), un costo (valor numérico) y un valor de *salt* (de hasta 16 bytes), donde la *salt* es típicamente un valor aleatorio. El resultado es un *hash* de 24 bytes en la forma:

```
$2<a/b/x/y>${cost}${22 character salt}[31 character hash]
```

¿Cómo se ve un hash de bcrypt?

Por ejemplo, para la clave `abc123xyz` con costo `12` y una *salt* aleatoria, el resultado es:

```
$2a$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW  
\_\_/\_/\_ \_-----/\_-----/  
Alg Cost      Salt                                     Hash
```

- `$2a$`: Identificador del algoritmo de *hashing* (bcrypt)
- `12`: Valor de costo (2^{12} , es decir, 4096 rondas)
- `R9h/cIPz0gi.URNNX3kh20`: Encoding de base-64 de la *salt* aleatoria ingresada
- `PST9/PgBkqquzi.Ss7KIUg02t0jWMUW`: Encoding en base-64 de los primeros 23 bytes del *hash* calculado de 24 bytes

¿Son seguras?

Si bien toma tiempo calcular el *hash* de una contraseña con bcrypt, también toma tiempo el “romperla”

Dado que bcrypt está diseñado para ser algo más lento que otros algoritmos, el factor de costo es pieza importante para aumentar la dificultad de “romper” el algoritmo

TIME TO CRACK: bcrypt Hashed Passwords



Number of characters	Numbers Only	Lowercase Only	Upper and Lower Case	Number, Upper, Lower	Number, Upper, Lower, Symbols
6	Instantly	7 minutes	7.5 hours	22 hours	11.5 days
7	Instantly	3 hours	16.2 days	8 weeks	3 years
8	3 minutes	4 days	2.4 days	9.5 years	286 years
9	23 minutes	2.8 months	120 years	583 years	27154 years
10	3.8 hours	6 years	6228 years	36160 years	2579596 years
11	38 days	161 years	323856 years	2241941 years	245061585 years
12	15 days	4169 years	16840527 years	139000337 years	23280850.6 thousand years
13	5.2 months	15483 years	875707453 years	8618021 thousand years	2211681 million years
14	4.3 years	2779344 years	45536787 thousand years	534317295 thousand years	210109676 million years
15	44 years	72262968 years	2367912 million years	33127672 million years	19960419.3 billion years
16	431 years	1878837183 years	123131474 million years	2053916 billion years	1896240 trillion years
17	4309 years	48849767 thousand years	6402837 billion years	127342773 billion years	180142784 trillion years
18	43084 years	1270094 million years	332947505 billion years	7895252 trillion years	17113565 quintillion years
19	430840 years	33022443 million years	17313271 trillion years	489505617 trillion years	1625789 quadrillion years
20	4308396 years	858583501 million years	900291 quintillion years	30349349 quintillion years	154449919 quadrillion years

Figura: [\[New research\] How tough is bcrypt to crack? And can it keep passwords safe?](#)

Coda Hale { writing, projects, about, contact }

How To Safely Store A Password

In which I recommend bcrypt.

31 Jan 2010

Use bcrypt

Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#). Use [bcrypt](#).

Why Not {MD5, SHA1, SHA2, SHA3, etc}?

These are all *general purpose* hash functions, designed to calculate a digest of huge amounts of data in as short a time as possible. This means that they are fantastic for ensuring the integrity of data and utterly rubbish for storing passwords.

A modern server can calculate the MD5 hash of about [330MB every second](#). If your users have passwords which are lowercase, alphanumeric, and 6 characters long, you can try *every single possible password of that size* in

[kelektiv/node.bcrypt.js - bcrypt for NodeJS](#):

Código y documentación de bcrypt para NodeJS, disponible en el repositorio oficial en GitHub

 README  MIT license  Security



node.bcrypt.js



A library to help you hash passwords.

You can read about [bcrypt in Wikipedia](#) as well as in the following article: [How To Safely Store A Password](#)

If You Are Submitting Bugs or Issues

Please verify that the NodeJS version you are using is a *stable* version; Unstable versions are currently not supported and issues created while using an unstable version will be closed.

If you are on a stable version of NodeJS, please provide a sufficient code snippet or log files for installation issues. The code snippet does not require you to include confidential information. However, it must provide enough information so the problem can be replicable, or it may be closed without an explanation.

Version Compatibility

Please upgrade to at least v5.0.0 to avoid security issues mentioned below.

Node Version

Bcrypt Version

Agenda

Identificación y autenticación

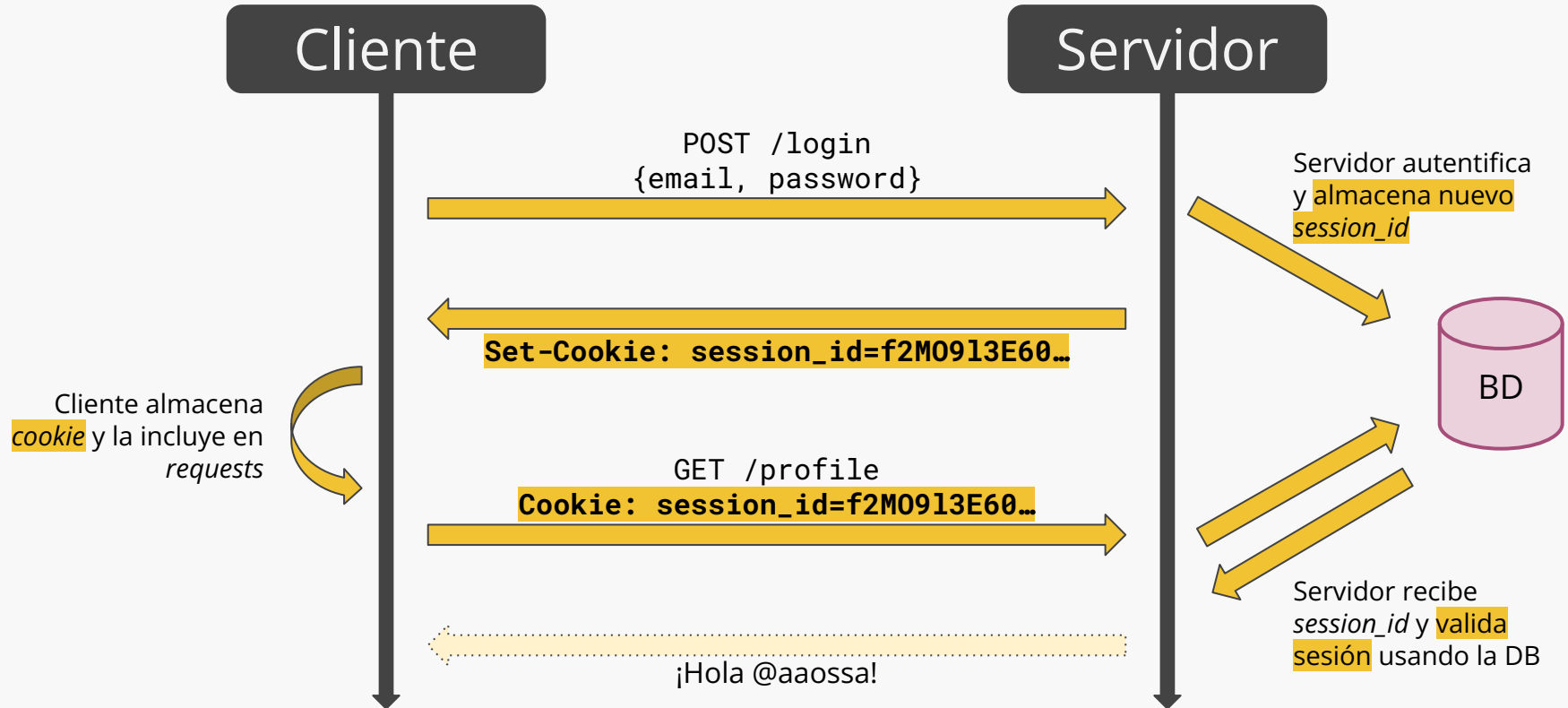
Hashing de contraseñas

✨ **JSON Web Tokens (JWT)**

Open Authorization (OAuth)

Single Sign-On (SSO)

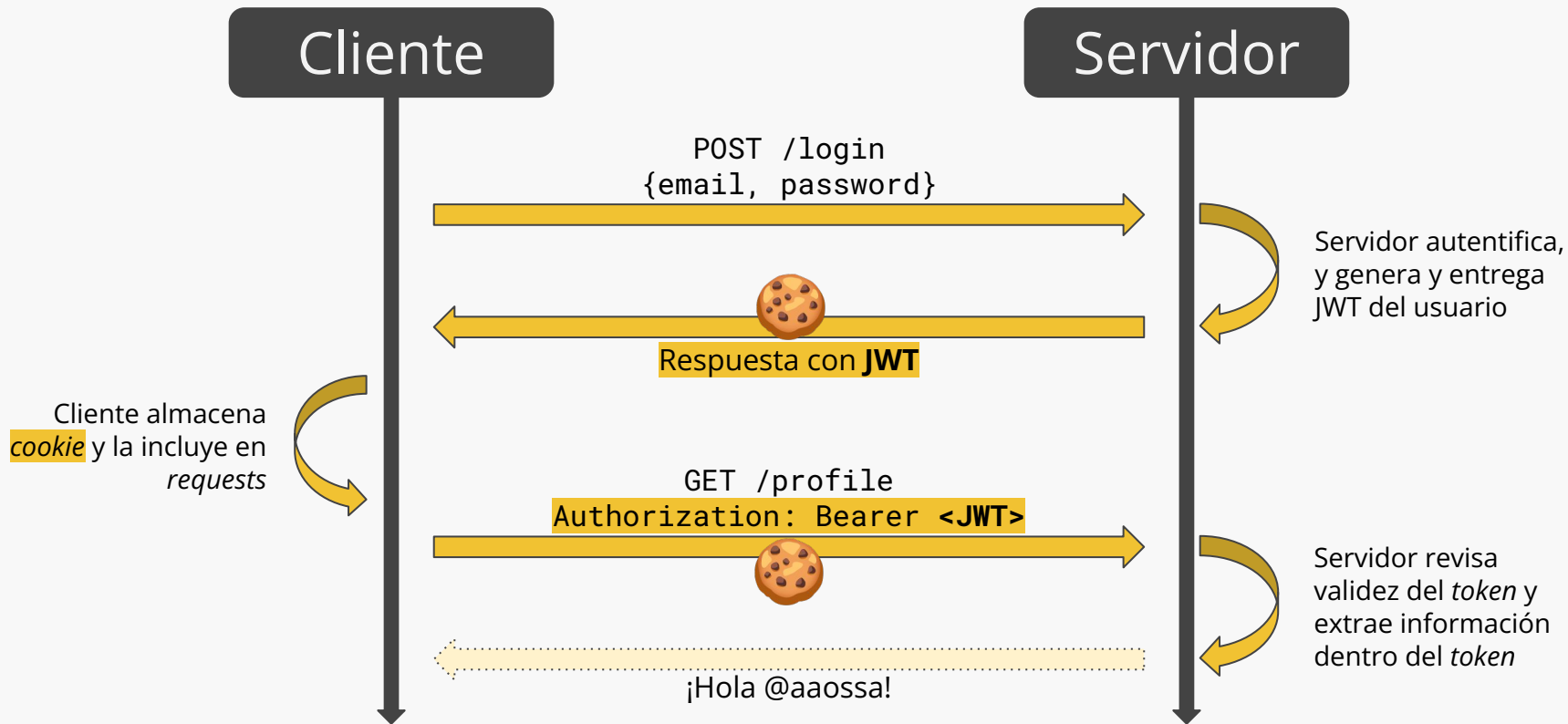
Funcionamiento de sesión de usuario



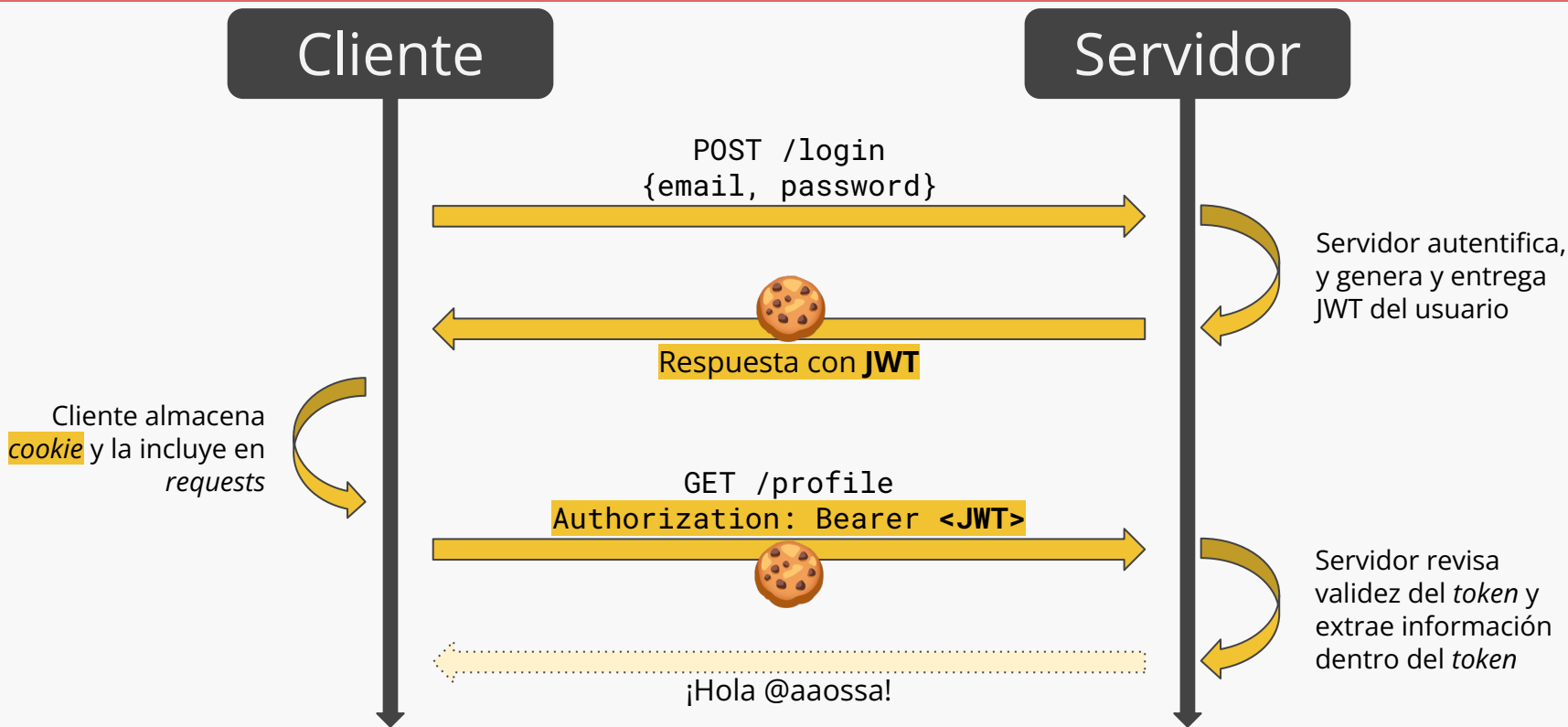
Funcionamiento de sesión de usuario



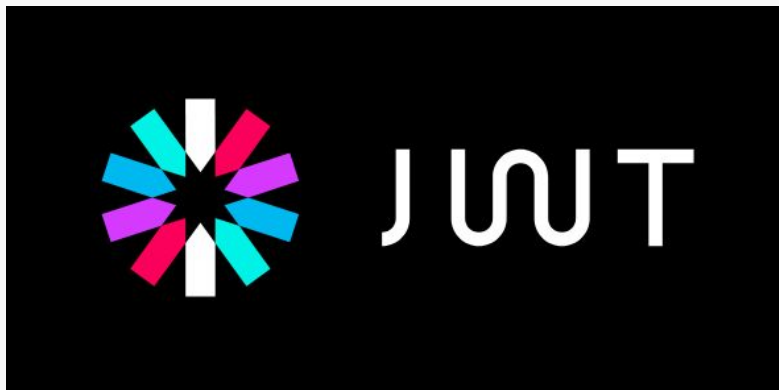
Autenticación con JWT



Pregunta#¿Qué diferencia ven con una sesión normal?



JWT: JSON Web Tokens



“JSON Web Token (JWT) is an open standard ([RFC 7519 - JSON Web Token \(JWT\)](#)) that defines a **compact** and **self-contained** way for securely transmitting information between parties as a **JSON object**.

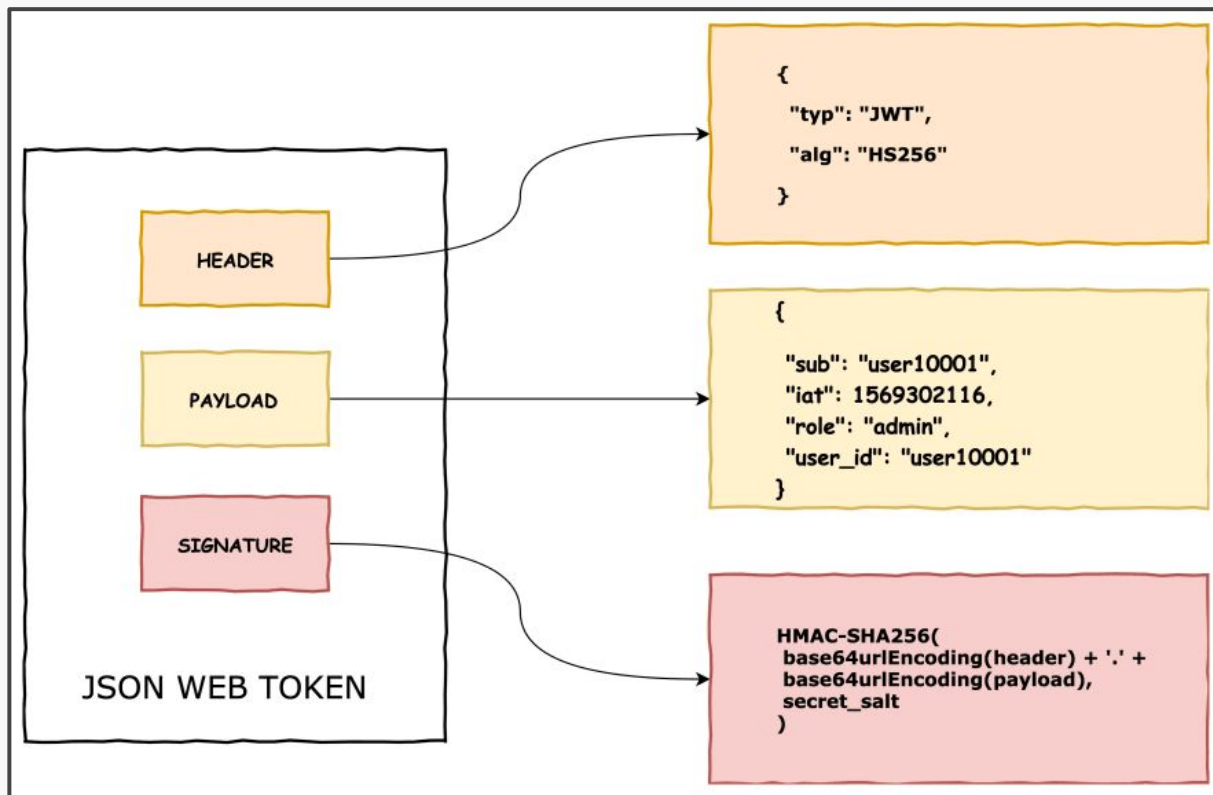
This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the [HMAC](#) algorithm) or a public/private key pair using [RSA](#) or [ECDSA](#).”

JWT: JSON Web Tokens



Estándar que permite **transmitir información** en un **objeto JSON** de forma segura entre dos entidades

Estructura de un JWT

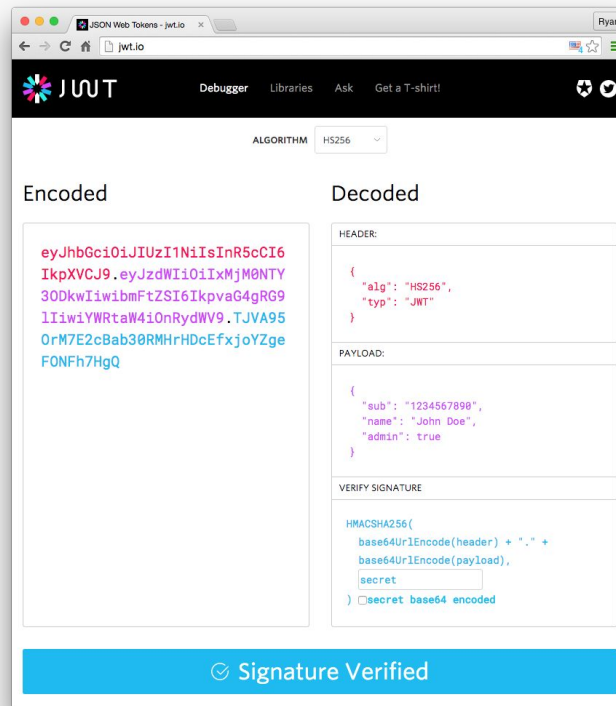


Estructura de un JWT

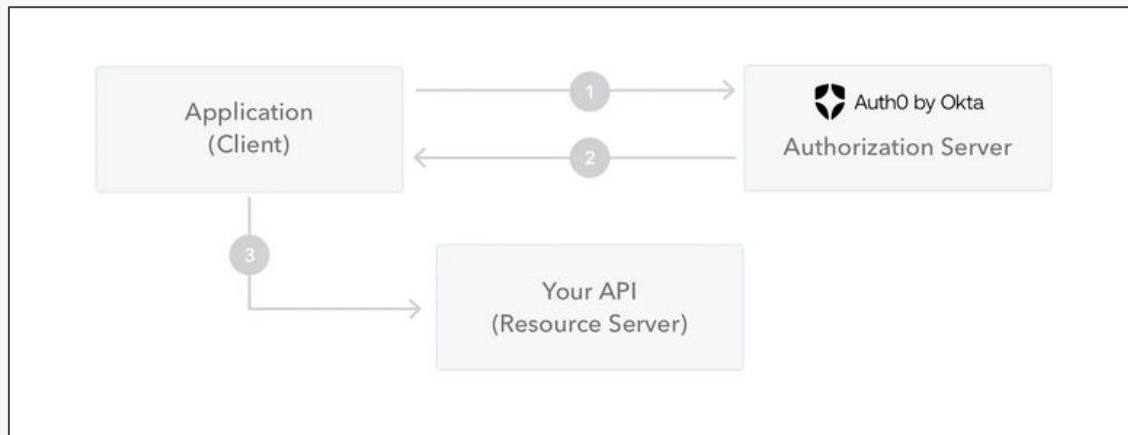
```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret)
```



¿Y cómo se usa?



1. La aplicación solicita autorización para acceder a un recurso al servidor de autorización (*authorization server*)
2. Cuando se entrega la autorización, el *authorization server* retorna un *access token* a la aplicación
3. La aplicación puede usar el *access token* para acceder a un recurso protegido (como una API)

El *token* puede contener información (por ejemplo, niveles de acceso) gracias a JWT

¿Y cómo se usa?

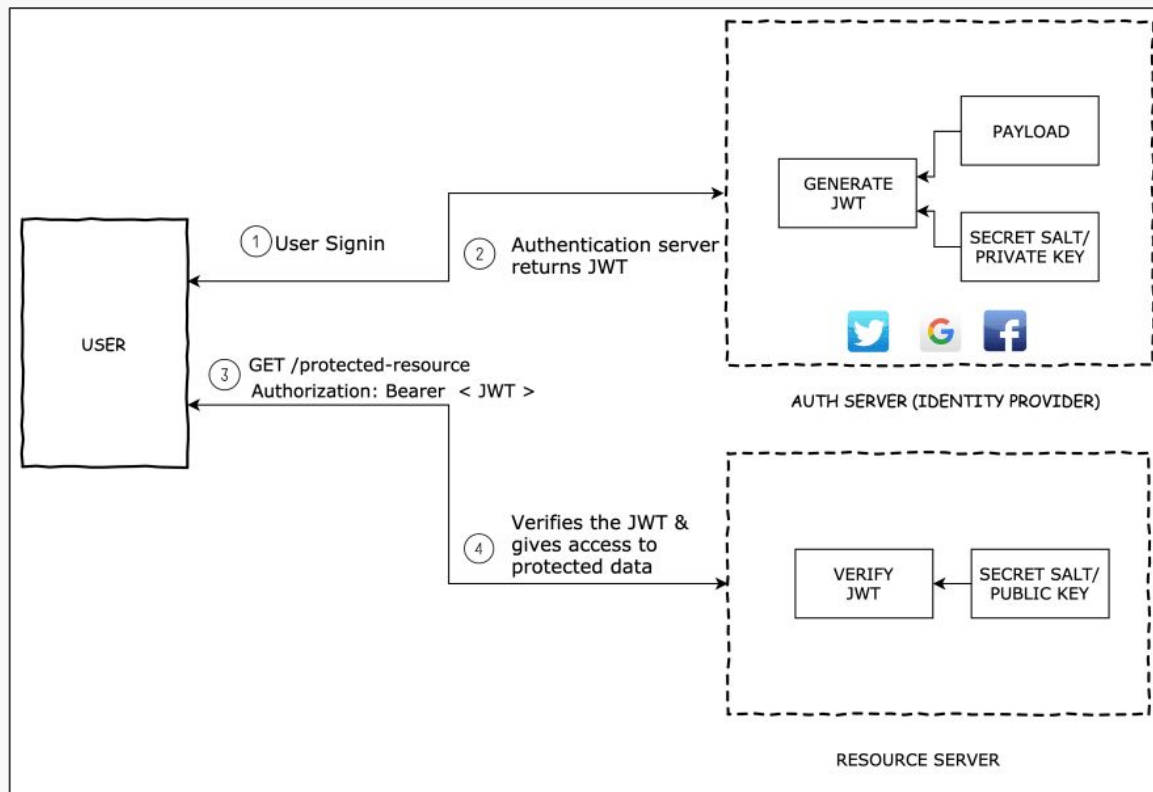
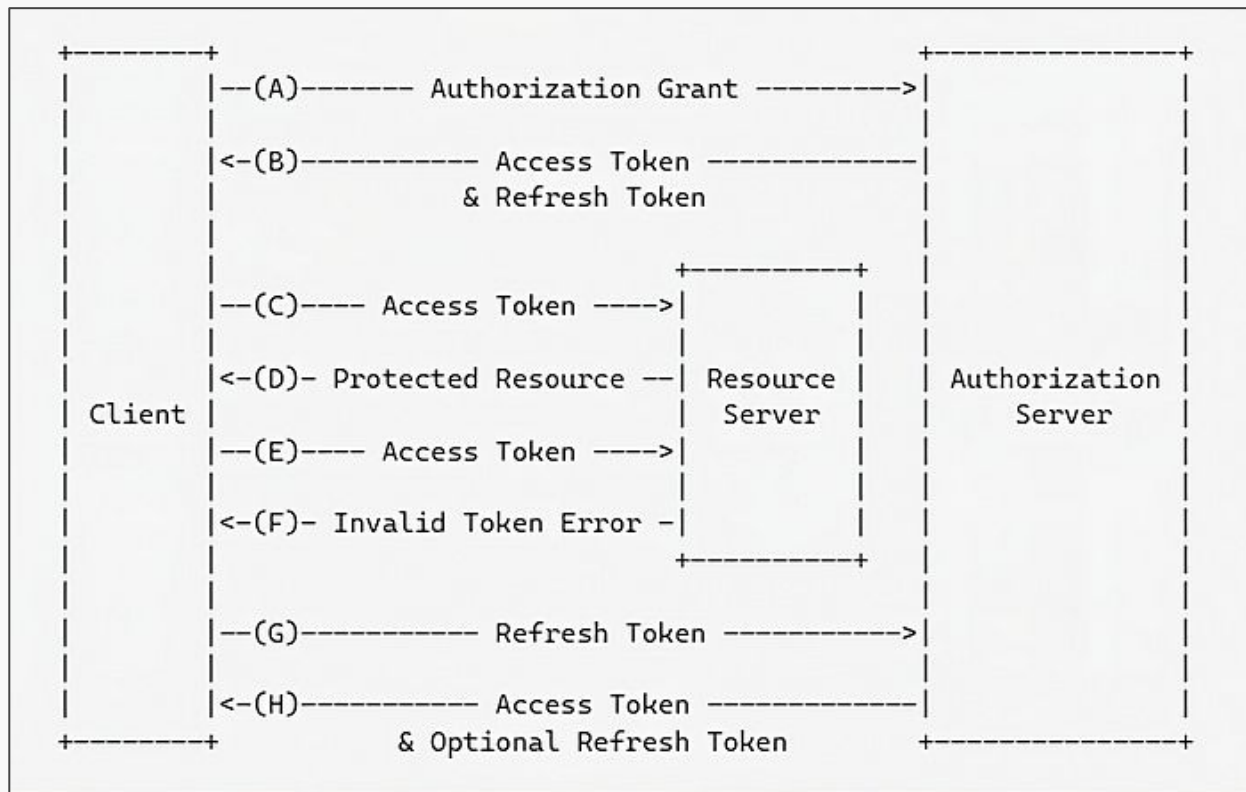


Figura: [How JWT \(JSON Web Token\) authentication works? - DEV Community](#)

¿Y cómo se usa?



¿Y para “cerrar sesión” (*sign out*)?

Al hacer *sign out*...

- Con *cookies* basta con borrar la *cookie* en el cliente, así el navegador no la envía más
- *Token* es almacenado en el *browser* por lo que podría seguir enviándolo a futuro
- Por lo tanto, hay que **“revocar” el *token***. Existen múltiples estrategias que no profundizaremos 👁👁
 - Usar *expiry time* relativamente corto
 - Hacer un *denylist* con los *tokens* que se deben invalidar
- Las estrategias sugeridas involucran “recordar” *tokens* (DB) o mantener sesiones activas más allá de lo debido, por lo que no es correcto usar, entonces... ¿cuál es el beneficio de usar JWT?

¿Y para “cerrar sesión” (*sign out*)?

Al hacer *sign out*...

- Con *cookies* basta con borrar la *cookie* en el cliente, así el navegador no la envía más
- ~~Tokens se almacenan en el navegador y la que no se olvida seguir enviándola a futuro~~

NO usar JWT como *session token**

- Usar *expiry time* relativamente corto
- Hacer un *denylist* con los *tokens* que se deben invalidar
- Las estrategias sugeridas involucran “recordar” *tokens* (DB) o mantener sesiones activas más allá de lo debido, por lo que no es correcto usar, entonces... ¿cuál es el beneficio de usar JWT?

Consideraciones de JWT

- Colocar **información precisa y breve** en *payload* ("compact")
- **No colocar información sensible** en el JWT porque es fácilmente decodificable (sino, hay que encriptar)
- Los JWT **no buscan encriptar ni ocultar información**, sino que hacen a la información "confiable", velando por la integridad y autenticidad de la información
- Los JWT están codificados y firmados, **NO encriptados**
- Para dar seguridad a la información de los *headers* de autorización, usamos **HTTPS**

Consideraciones de JWT

- Colocar información precisa y breve en *payload* ("compact")

NO usar JWT como *session token**

"confiable", velando por la integridad y autenticidad de la información

- Los JWT están codificados y firmados, NO encriptados
- Para dar seguridad a la información de los *headers* de autorización, usamos HTTPS

Consideraciones de JWT

- Colocar información precisa y breve en *payload* ("compact")

Se recomienda usar en APIs o arquitecturas de auth más complejas (como OAuth)

"confiable", velando por la integridad y autenticidad de la información

- Los JWT están codificados y firmados, NO encriptados
- Para dar seguridad a la información de los *headers* de autorización, usamos HTTPS

¿Cómo hacerlo en Express?

Utilizamos jsonwebtoken
para crear *tokens* (una vez
que se haga *login*):

```
const jwt = require('jsonwebtoken');
const { expressjwt: expressJwt } = require('express-jwt');
const JWT_SECRET = process.env.JWT_SECRET;

app.post('/login', (req, res) => {
  // ...

  // Create payload (no sensitive data)
  const payload = { sub: user.id, username: user.username };

  // Sign the token (expires in 1 hour)
  const token = jwt.sign(payload, JWT_SECRET, { expiresIn: '1h' });

  res.json({ token });
});
```

¿Cómo hacerlo en Express?

Utilizamos express-jwt para proteger rutas, las que serán accesibles solo usando JWTs generados por nosotros

```
const jwt = require('jsonwebtoken');
const { expressjwt: expressJwt } = require('express-jwt');
const JWT_SECRET = process.env.JWT_SECRET;

// Protect routes with express-jwt
const authenticate = expressJwt({
  secret: JWT_SECRET,
  algorithms: ['HS256'],
  requestProperty: 'auth' // Payload available in req.auth
});

app.get('/protected', authenticate, (req, res) => {
  res.json({
    message: `Hello ${req.auth.username}`,
    userId: req.auth.sub
  });
});
```

¿Cómo hacerlo en Koa?

Utilizamos jsonwebtoken
para crear *tokens* (una vez
que se haga *login*):

```
const jwt = require('jsonwebtoken');
const koaJwt = require('koa-jwt');
const JWT_SECRET = process.env.JWT_SECRET;

router.post('/login', async (ctx) => {
  // ...

  // Create payload (no sensitive data)
  const payload = { sub: user.id, username: user.username };

  // Sign the token (expires in 1 hour)
  const token = jwt.sign(payload, JWT_SECRET, { expiresIn: '1h' });

  ctx.body = { token };
});
```

¿Cómo hacerlo en Express?

Utilizamos express-jwt para proteger rutas, las que serán accesibles solo usando JWTs generados por nosotros

```
const jwt = require('jsonwebtoken');
const koaJwt = require('koa-jwt');
const JWT_SECRET = process.env.JWT_SECRET;

// Protect routes with koa-jwt
const authenticate = koaJwt({
  secret: JWT_SECRET,
  algorithms: ['HS256']
});

router.get('/protected', authenticate, async ctx => {
  const user = ctx.state.user;
  ctx.body = {
    message: `Hello ${user.username}`,
    userId: user.sub,
  };
});
```

¿Diferencia entre JWT y *session*?

- ***client-side vs. server side***, es decir, JWT vs. sesiones:
 - JWT no ofrece un beneficio de por sí sobre las sesiones, la única diferencia es dónde se mantiene el *session state*
 - Con sesiones, el estado de la sesión se almacena en la DB o en la memoria del servidor, lo que tiene ventajas y desventajas
 - Con JWT, la alternativa es pasar al cliente ese almacenamiento y usar el token/cookie/secret para contener información de la sesión
- ***cookies vs. localStorage***:
 - Una pregunta de implementación común suele ser elegir entre almacenar el JWT en localStorage o usar una *cookie*
 - NUNCA usar localStorage para almacenar tokens (TLDR: tiene baja performance y, peor aún, es accesible por JavaScript)

Recursos adicionales de la clase

- [Secure Your Node.js and Express.js API with JWT Authentication](#)
- [Secure a Koa API With a JWT Token](#) (2016)

Otros recursos relacionados:

- [Encoding and Decoding Base64 Strings in Python](#): Cómo funciona codificar en base64
- [Secure Hash Algorithms | Brilliant Math & Science Wiki](#): Explicación de cómo funcionan los algoritmos SHA (tales como SHA-1 usado por Git)
- [How It Works - Let's Encrypt](#): Cómo funciona Let's Encrypt (certificados HTTPS gratuitos)
- [Auth0 - Servicio de autenticación](#): Plataforma de autenticación y autorización adaptable y fácil de instalar

Agenda

Identificación y autenticación

Hashing de contraseñas

JSON Web Tokens (JWT)

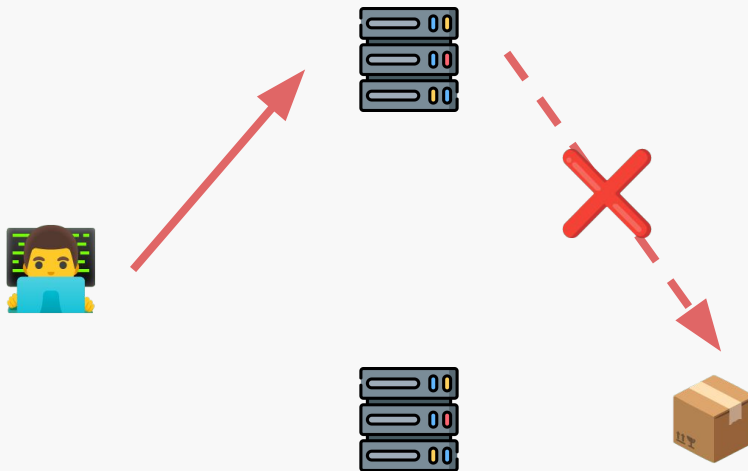
✨ **Open Authorization (OAuth)**

Single Sign-On (SSO)

OAuth

Supongamos que queremos permitir una aplicación acceda a recursos nuestros en otra aplicación

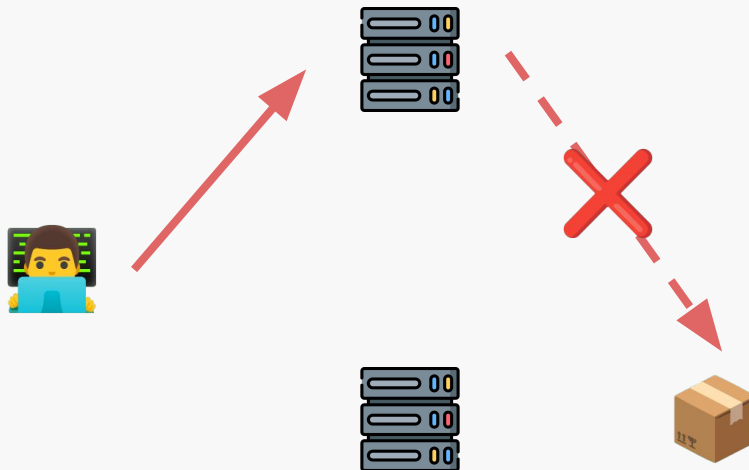
Sin embargo, esa aplicación no tiene la autorización necesaria para acceder al recurso ni puede autenticarse en nuestro nombre



OAuth

La solución más trivial sería darle nuestra contraseña para que haga *requests* a nuestro nombre

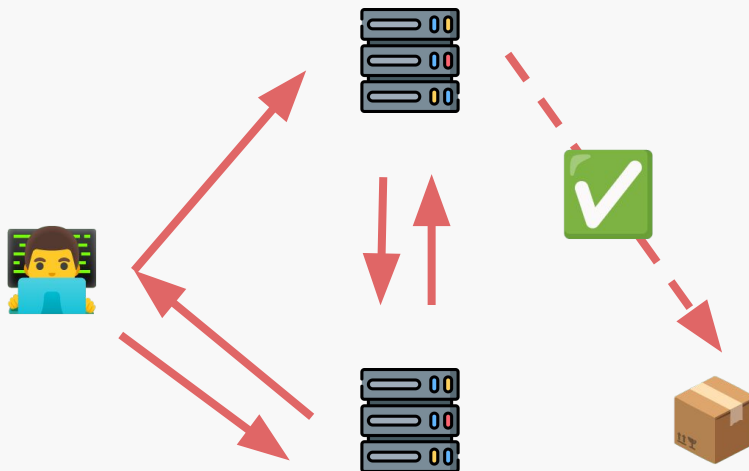
¿Hay una solución mejor?



OAuth

La respuesta es que sí existe tal mecanismo, definido por medio de un estándar, que sincroniza a usuario y aplicaciones para que estas puedan acceder a recursos en nuestro nombre

Este estándar es es [OAuth 2.0](#)



OAuth



“OAuth 2.0 is the industry-standard protocol for authorization. OAuth 2.0 focuses on client developer simplicity while providing specific authorization flows for web applications, desktop applications, mobile phones, and living room devices. This specification and its extensions are being developed within the [IETF OAuth Working Group](#).”

OAuth

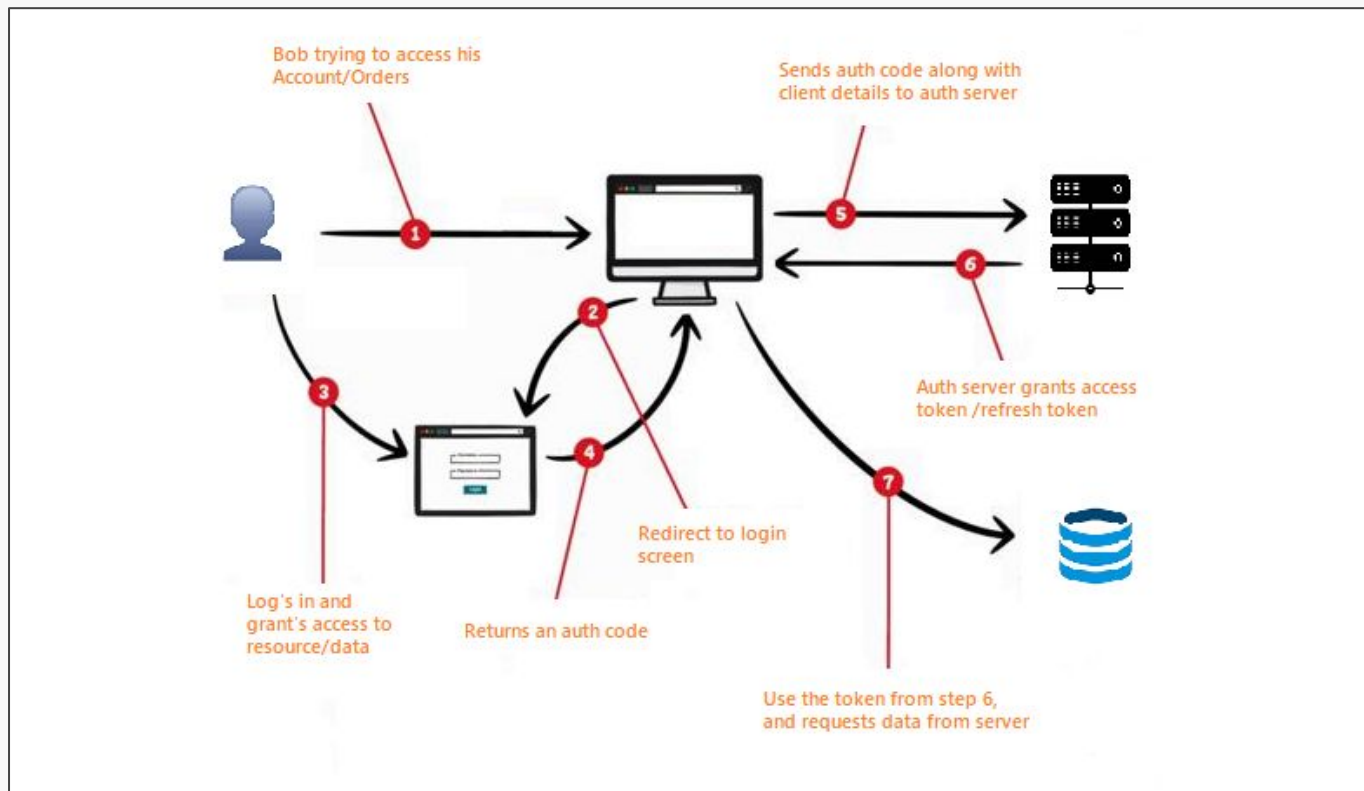


Figura: [Understanding OAuth 2.0: A Step-by-Step Guide | CodeNx](#)

OAuth

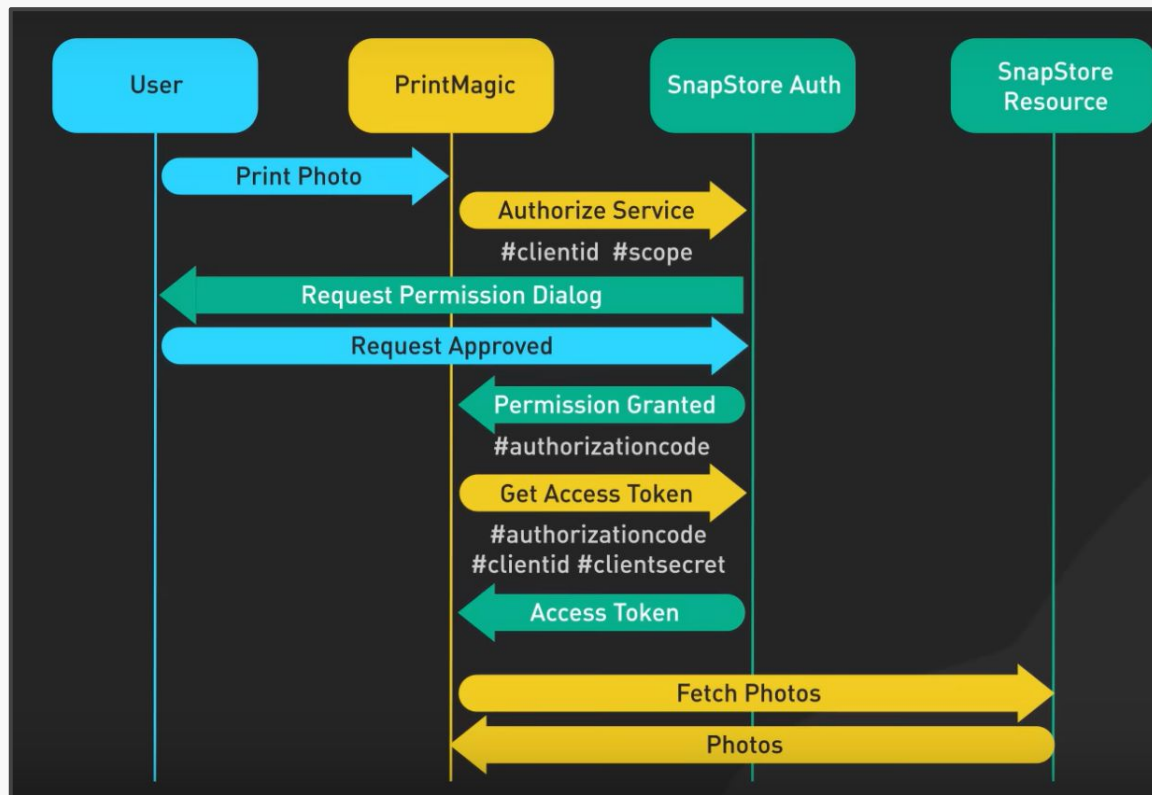


Figura: [OAuth 2 Explained In Simple Terms - Video de "ByteByteGo" en YouTube](#)

Agenda

Identificación y autenticación

Hashing de contraseñas

JSON Web Tokens (JWT)

Open Authorization (OAuth)

✨ **Single Sign-On (SSO)**

¿Cómo implementar *Single Sign-On*?

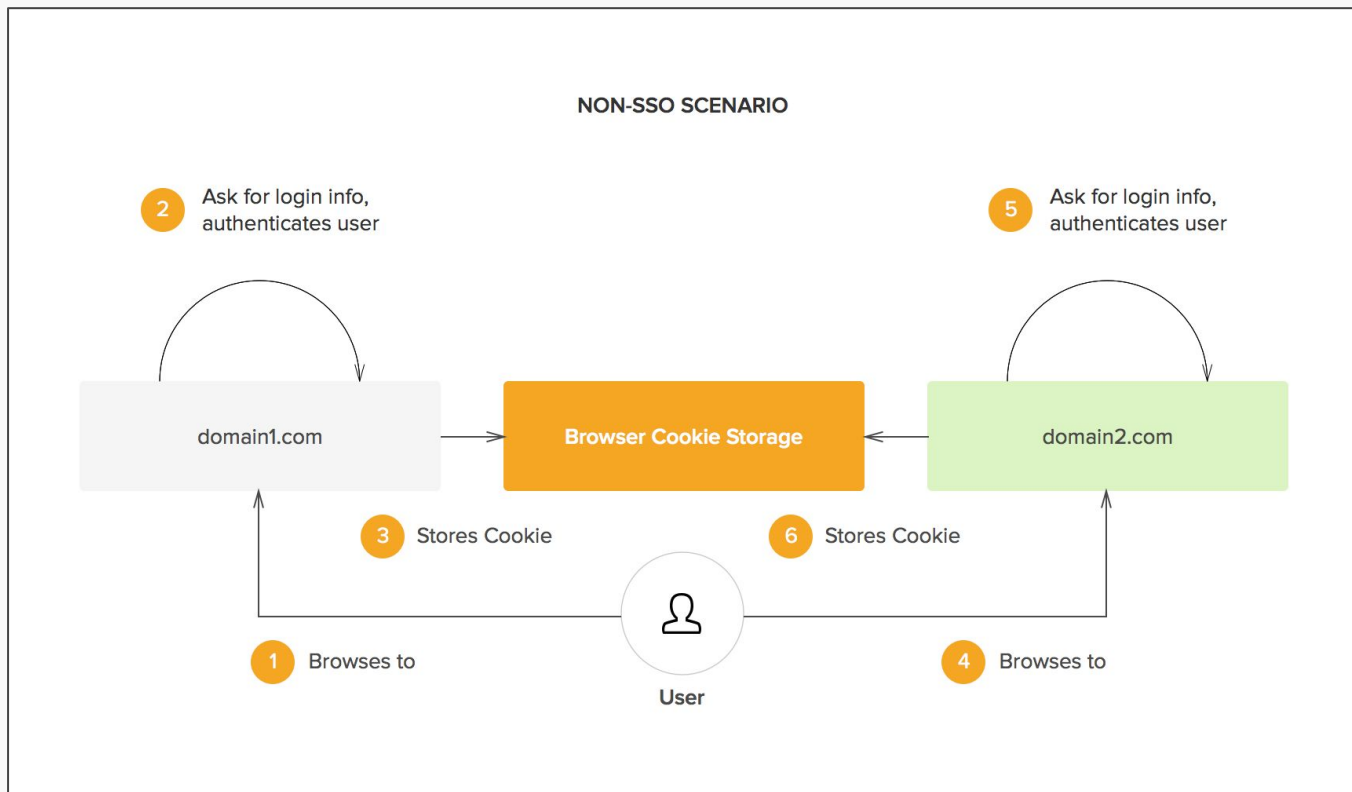


Figura: [What Is Single Sign-On Authentication \(SSO\) And How Does It Work?](#)

¿Cómo implementar *Single Sign-On*?



Pregunta# ¿Puedo usar la misma *cookie* en dos sitios?

¿Cómo implementar *Single Sign-On*?

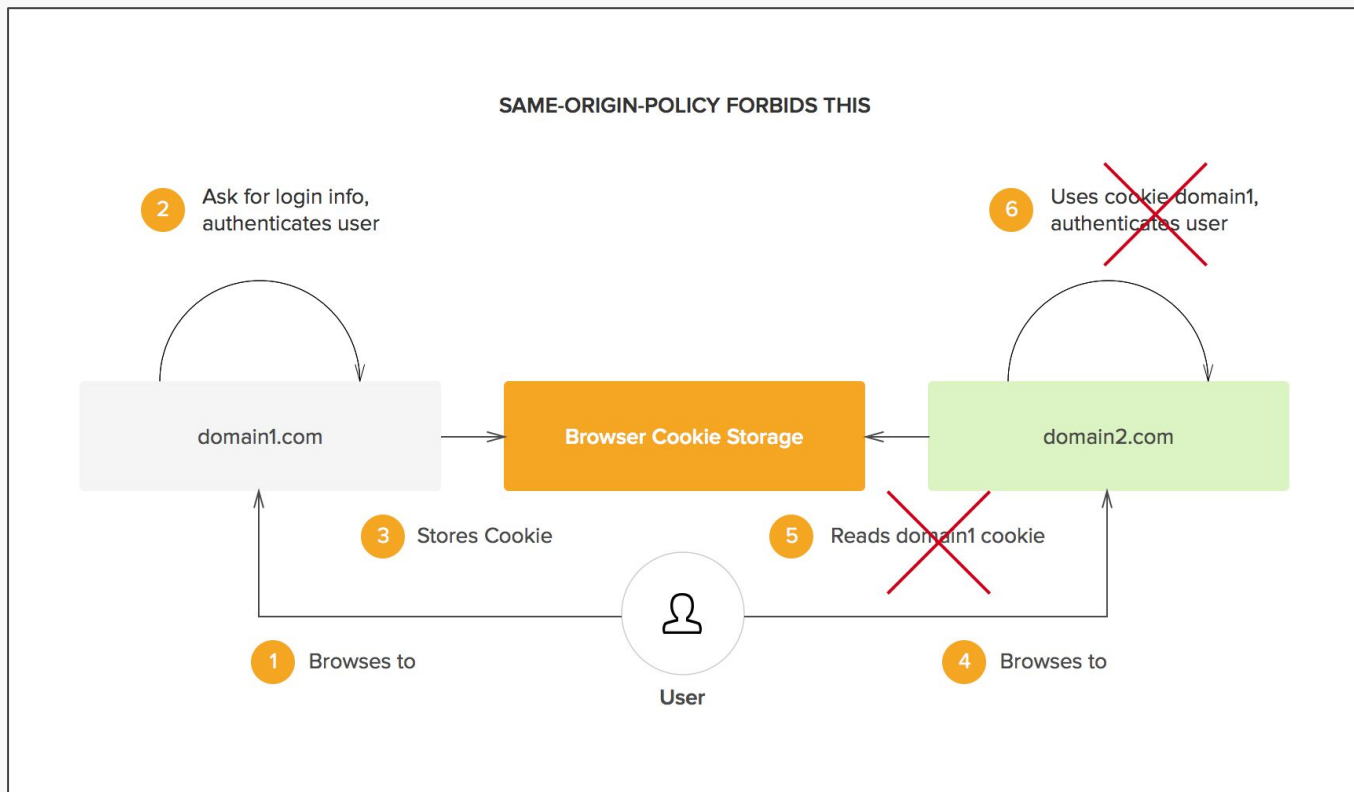


Figura: [What Is Single Sign-On Authentication \(SSO\) And How Does It Work?](#)

¿Cómo implementar *Single Sign-On*?

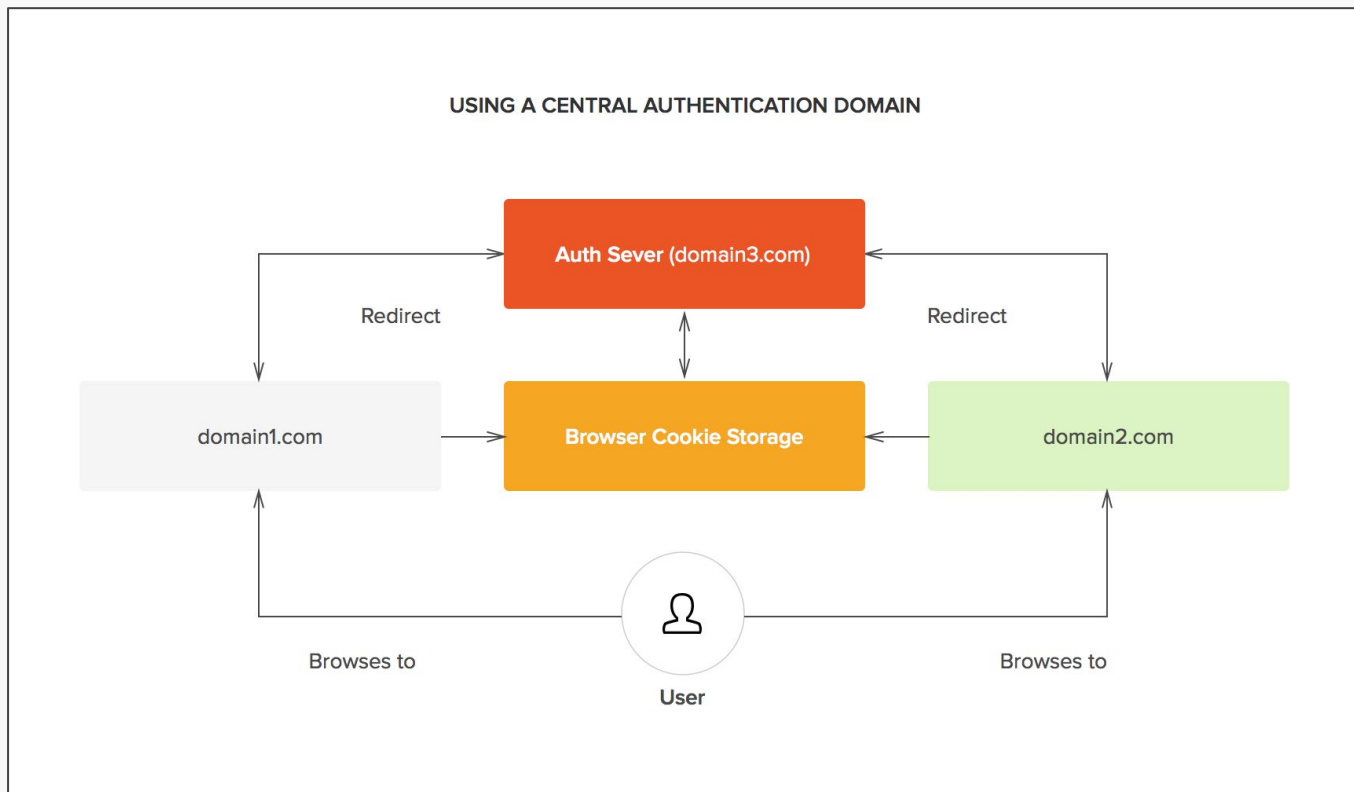


Figura: [What Is Single Sign-On Authentication \(SSO\) And How Does It Work?](#)

¿Cómo implementar *Single Sign-On*?

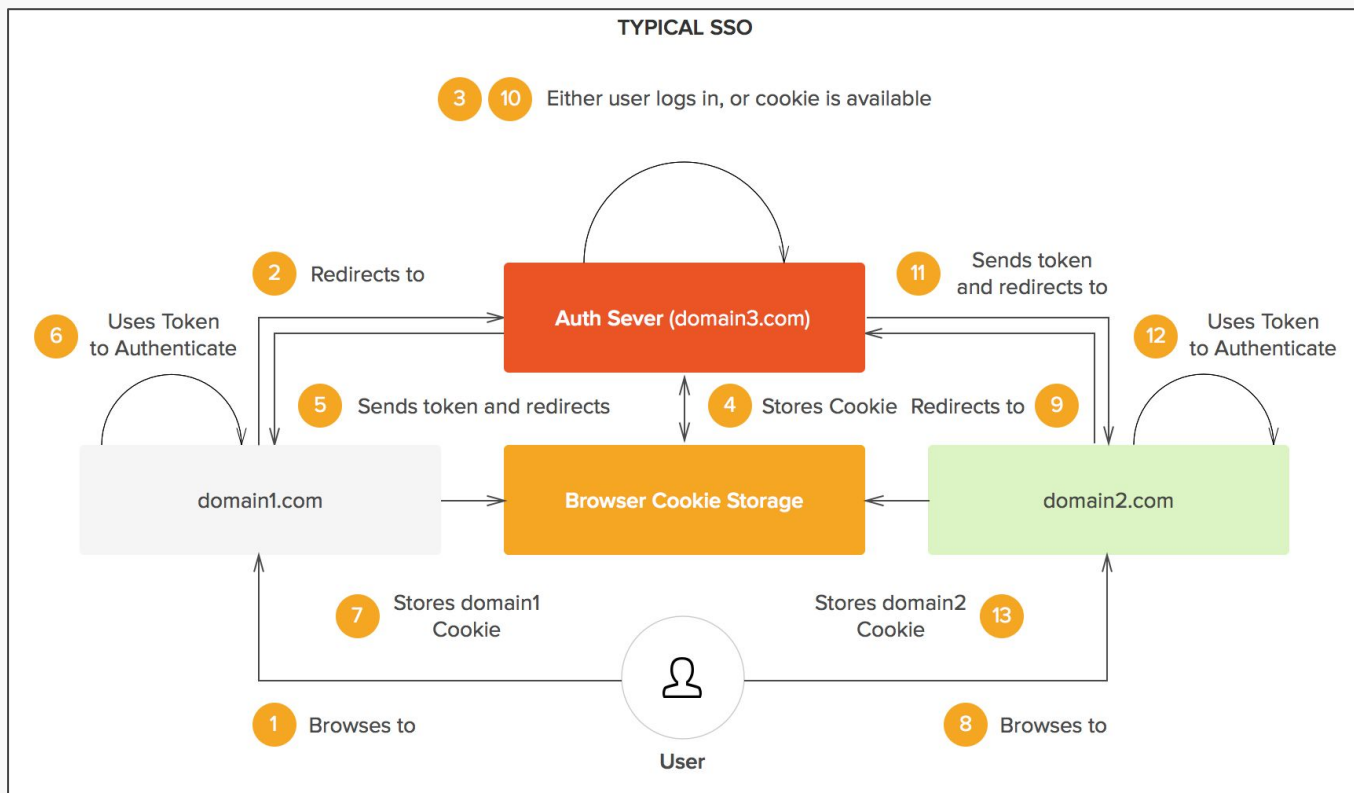


Figura: [What Is Single Sign-On Authentication \(SSO\) And How Does It Work?](#)

Agenda

Identificación y autenticación

Hashing de contraseñas

JSON Web Tokens (JWT)

Open Authorization (OAuth)

Single Sign-On (SSO)



¿Qué se viene pronto?

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 10 (05/5 - 11/5)		Lanzamiento proyecto (E0)		Post-receso y repaso			
Semana 11 (12/5 - 18/5)		Cookies y sessions		HOY			
Semana 12 (19/5 - 25/5)		Charla GraphQL (Daniela Flores)	Día de las Glorias Navales	Almacenamiento (cookies, tokens, IndexDB, etc)			
Semana 13 (26/5 - 01/6)		Tecnologías adicionales		Testing	Sala de ayuda: E1		
Semana 14 (02/6 - 08/6)		Seguridad web		Protocolos seguros			

Próximas fechas importantes: evaluaciones

	Evaluaciones							
	Lunes	Martes	Miércoles	Jueves	Viernes		Sábado	Domingo
Semana 10 (05/5 - 11/5)	UC: Sin eval.	Inicio E0						
Semana 11 (12/5 - 18/5)				HOY	Fin E0	Inicio E1		
Semana 12 (19/5 - 25/5)	Control 3		Día de las Glorias Navales					
Semana 13 (26/5 - 01/6)					Fin E1	Inicio E2		
Semana 14 (02/6 - 08/6)	Control 4							

Clase 15

Autenticación y autorización

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl